



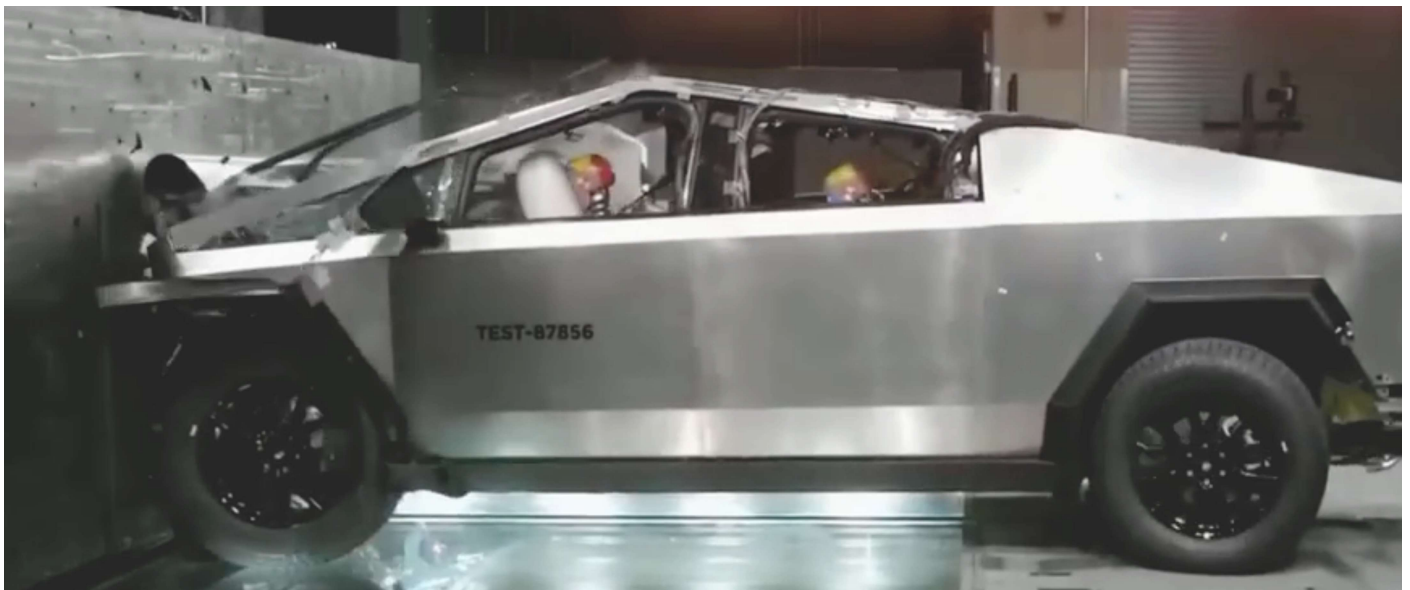
EGH456 ASSESSMENT 2

REPORT

Embedded System for Electric Vehicle

Group 16

Joel Kemp
Ben Cikes
Rory Blair
Liam Mills



7/06/2025

Table of Contents

Table of Contents	I
List of Tables & Figures.....	II
1). Introduction	1
2). Design & Implementation	2
2.1). Motor Control	2
2.1.1). Hall Sensor Interrupt	3
2.1.2). Motor Task	3
2.2). Sensing.....	4
2.2.1). Motor Power Sensing.....	4
2.2.2). Speed Sensing	5
2.2.3). Light Sensing.....	5
2.2.4). Vehicle Body Acceleration Sensing	6
2.3). User Interface.....	6
2.3.1). Approach to Design:.....	6
2.3.2). RTOS Task Structure, Task Communication & Synchronization:	7
2.3.3). Important Issues and Choices:.....	8
2.3.4). Drivers, Modules, and External Libraries:	8
2.3.5). Relationship to Hardware and Software Platforms:	8
3). Results	9
3.1). Motor Control	9
3.2). Sensing.....	10
3.2.1) Power sensor Filtering.....	10
3.2.2) Speed Sensor Filtering	11
3.2.3) Light Level Sensor	11
3.2.4) Vehicle Acceleration Sensor	12
3.3). User Interface.....	12
4). References	14

List of Tables & Figures

TABLE 1.1: MEMBER CONTRIBUTION	1
FIGURE 2.1: RTOS-BASED MOTOR CONTROL FLOWCHART SHOWING HOW HALL INTERRUPTS, TIMER-DRIVEN SPEED SENSING, AND CONDITIONAL E-STOP LOGIC FEED INTO DUTY CYCLE ADJUSTMENT VIA SPEED CONTROL.	2
TABLE 2.1: MOTOR SYSTEM RTOS COMPONENTS AND ROLES.....	3
FIGURE 2.2: DIAGRAM OF THE RTOS POWER SENSING, SHOWING THE SEQUENCE FROM ADC MOTOR SAMPLING TASKS TO FILTER PROCESSING AND FREERTOS QUEUE	4
FIGURE 2.3: DIAGRAM OF THE RTOS SPEED SENSING, SHOWING THE SEQUENCE FROM HALL SENSOR INTERRUPTS TO MOTOR SYSTEM VIA TIMER ISR AND FREERTOS QUEUE	5
FIGURE 2.4: PANEL LAYOUT	7
FIGURE 2.5: RTOS TASK STRUCTURE.....	7
FIGURE 3.1: SERIAL PLOT SHOWING ACCELERATION TO 2000 RPM AND ERROR BEFORE E-STOP ACTIVATION. NOTE: X-AXIS IS IN 100 MS SCALE (100 = 10S).	9
FIGURE 3.2: SERIAL PLOT SHOWING DECELERATION AFTER E-STOP TRIGGER.....	9
FIGURE 3.3: COMPARISON OF UNFILTERED (RED) AND FILTERED (BLUE) POWER READINGS.	10
FIGURE 3.4: COMPARISON OF UNFILTERED (RED) AND MOVING-AVERAGE FILTERED (BLUE) RPM ESTIMATES.....	11
FIGURE 3.5: REAL-TIME PLOT OF RAW AND FILTERED OPT3001 LIGHT MEASUREMENTS WITH VISUALISED DAY/NIGHT TRANSITIONS AT THRESHOLD=50 LUX.	11
FIGURE 3.6: WORKING PLOTS PANEL	12

1). Introduction

Due to their environmental benefits and efficiency, there has been a shift in the automotive trade towards Electric Vehicle (EV). This project, undertaken by a team of embedded system engineers, addresses the development of real-time software for an EV using the Tiva TMC1294NCPDT microcontroller. The objective is to establish safe motor controls, sensor monitoring, and user interaction via an interface for an EV system. EVs rely on modern embedded systems to supervise motor control, sensing and user interface (UI). Commissioned by an automotive manufacturer transitioning from older petrol cars to EVs, this project employs the use of the Tiva microcontroller and Free Real-Time Operating System (FreeRTOS) to manage a BLDC motor, monitor important variables, such as power and speed, and create a user-friendly UI.

The embedded system is comprised of three subsystems, with the motor control requiring a DRV832x driver to control the motor speed (RPM), support safe acceleration and deceleration and initiate E-Stops based on current and acceleration thresholds while transmitting data via UART. The sensing subsystem includes drivers for motor power (≤ 150 Hz), speed (≥ 100 Hz), Light (≥ 2 Hz) and Acceleration (≥ 100 Hz), with data filtered outside ISRs and output via UART. The UI involves an interface that includes features such as motor start/stop, RPM slider, real-time clock, thresholds for current and acceleration, status indicator, E-stop acknowledgment, Day/Night indicator and plots for each sensor output. All members and their contribution to this project are in Table 1.1.

Members	Contribution
Joel Kemp	Motor Control, Light Sensing & Subsystem Integration
Ben Cikes	Power Sensing, Speed Sensing & Subsystem Integration
Liam Mills	Acceleration Sensing
Rory Blair	User Interface, Light Sensing & Subsystem Integration

Table 1.1: Member Contribution

This project delivers an embedded EV system on a Tiva microcontroller, integrating motor control, sensing and a UI, with FreeRTOS. This will ensure real-time operation through modular drivers allowing safe motor control, clean and filtered sensor data, and an interactive UI, which would satisfy the automotive manufacturers requirements.

2). Design & Implementation

2.1). Motor Control

The motor subsystem was developed to safely start, stop, and control a BLDC motor in response to user-specified RPM values. The motor subsystem had two key requirements:

- **Speed Control** to a user-requested desired speed in revolutions per minute (RPM) within an appropriate control error of 20 %.
- **Safe acceleration bounds:** ≤ 500 RPM/s during normal acceleration or deceleration and exactly 1000 RPM/s for an emergency stop.

The design utilised the MotorLib library to manage low-level motor control to reduce the risk of hardware damage.

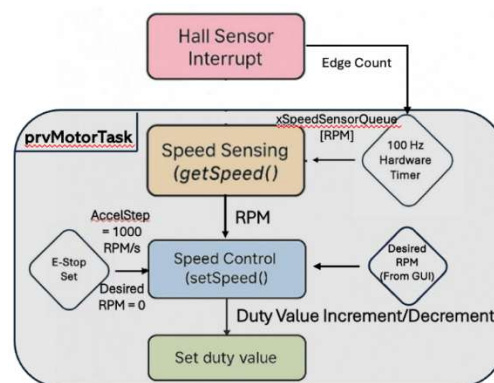


Figure 2.1: RTOS-based motor control flowchart showing how Hall interrupts, timer-driven speed sensing, and conditional E-stop logic feed into duty cycle adjustment via speed control.

The implementation of these functionalities is simplified in figure 2.1, which outlines the interaction of the four key components of the motor system: Hall Sensor/Edge count, speed sensing, speed control, and duty value setting.

The task and interrupt structure followed a flat RTOS architecture. First the hall sensor ISR was created to read the hall sensor GPIO's and trigger on both rising and falling edge. This interrupt increments a global edge count variable `g_edgeCount_cur` and invokes motorlib API: `updateMotor()`. Every 10 ms, the Timer3A ISR (100Hz) captures the edge count and pushes it into `xSpeedSensorQueue`.

The motor system utilised only one main Motor Task, which begins by starting the motor. This initialises the motor at a PWM duty period of 50 using the motorlib function `initMotorLib()`, and sets the duty value (using motorlib API `setDuty()`) at $> 10\%$ of the period to spin the motor.

It then checks if the speed sensor queue is available to receive from, and if so, runs every 100 ms at `priority = idle + 1`. First it receives the moving average speed via `getSpeedMotor()`. It then attempts to take `xDesiredRpmSemaphore`, which is shared with the GUI display task and indicates the update of user-set desired speed. If successful, the current duty cycle is adjusted toward the new RPM target, via `updateSpeedMotor()`.

RTOS Component	Role
Hall Sensor ISR	Increment edge count, call <code>updateMotor()</code>
Hardware Timer3A (10 ms)	Push edges per 10 ms into <code>xSpeedSensorQueue</code>
MotorTask (100 ms)	Compute RPM, adjust duty, print RPM via UART
Synchronisation	Queue: <code>xSpeedSensorQueue</code> Semaphores: <code>xDesiredRpmSemaphore</code>

Table 2.1: Motor System RTOS Components and Roles

2.1.1). Hall Sensor Interrupt

Motor state feedback was obtained via three integrated digital Hall effect sensors, connected to GPIO pins PM3, PH2, and PN2. These sensors detect the motor's position by responding to the magnetic field transitions of the motor's internal magnets. As the motor spins, the sensors produce a specific sequence of high and low digital signals corresponding to specific angular regions. To infer speed, an interrupt **HallSensorInterrupt()** was created to log each rising or falling edge on the Hall lines (representing a transition from one hall state to the next), which was counted over each period, resulting in a count of how many transitions occurred during that period. It was manually tested and verified that the motor produces a 24 of these transitions per revolution, allowing interpretation of speed, based on this periodic count. This will be discussed in the speed sensing section (2.2.2).

2.1.2). Motor Task

Speed control was implemented using a simple ramping algorithm. The **updateSpeedMotor()** function adjusted the PWM duty cycle based on the error between the current RPM and user-set desired RPM values, received from a semaphore shared with the Display task. The function logic first evaluates whether the current RPM lies outside a bound defined by *accelStep* and *decelStep* around the desired setpoint. If the current RPM is lower than the desired, an acceleration routine is triggered. Here:

- Desired RPM is incremented by *accelStep* (25 RPM each 100 ms period, yielding 250 RPM/s).
- A counter accumulates consecutive iterations to align this *accelStep* with the duty value's impact on RPM. Because a PWM period value of 50 was used, a single increment of the duty value results in a 2% increase in speed. It was validated that the motor achieves a speed of $\approx 5000\text{RPM}$ at max duty value. Therefore, this 2% increase corresponds to an increase in RPM of 100. Therefore, when this counter reaches 4, the duty value is incremented ($4\text{iterations} \times 25\text{RPM} = 100\text{RPM}$).
- The function then calls the motorlib API function `setDuty()`, using the incremented duty value, to affect the desired speed increase.

The same logic applies to deceleration handling, which decrements the duty value to induce deceleration at the same rate, and according to the same (but inverse) conditions.

The planned design included a e-stop functionality via semaphore signalling from either the power or acceleration sensing tasks. However, due to integration limitations, this logic was not fully connected. Instead, a surrogate timer-based trigger (`estop_counter`) was used to trigger a simulated E-stop.

When triggered, a separate function (`updateSpeedEstop()`) executed a linear ramp-down of speed. This function mirrored the logic used in normal deceleration but with a fixed and more aggressive rate. Specifically:

- The function forcibly reduced the current RPM setpoint by 100 RPM per iteration (every 100 ms), equivalent to 1000 RPM/s.

- The duty value was decremented and set via the motorlib API function setDuty().
- The process continued until the RPM reached a threshold (250 RPM), which triggered the call of motorlib API stopMotor(), bringing the motor to a safe stop.

2.2). Sensing

2.2.1). Motor Power Sensing

The motor current sensing system was implemented using analog signal conditioning and FreeRTOS-based concurrent tasks. The system samples two motor phase currents—specifically, phase B and phase C—via ADC inputs AIN5 (PD2) and AIN0 (PE3). These currents are first amplified by the DRV8323 gate driver’s internal current sense amplifiers, then digitised using ADC1 on the Tiva C microcontroller.

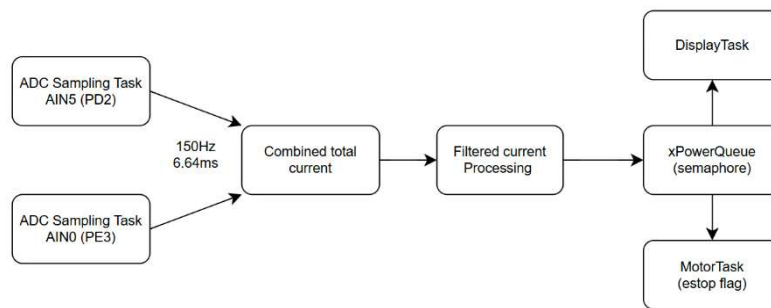


Figure 2.2: Diagram of the RTOS power sensing, showing the sequence from ADC motor sampling tasks to filter processing and FreeRTOS queue

Current sensing was achieved through two dedicated FreeRTOS tasks, each responsible for sampling one phase. `prvCurrentSenseTask_AIN5` handles input from PD2 while `prvCurrentSenseTask_AIN0` handles PE3. Both tasks use `vTaskDelayUntil()` to maintain a fixed sampling period of 6.64 ms, corresponding to a 150 Hz sampling frequency. This timing was chosen to satisfy task criteria.

Each task reads from its respective ADC channel and converts the raw 12-bit ADC values to current using a linear transformation derived from the system’s analogue front-end:

$$I = \frac{(V_{adc} - V_{mid})}{(Gain \times R_{shunt})}$$

where V_{adc} is the measured voltage, V_{mid} is mid-supply reference ($V_{REF} / 2$), $Gain$ is the amplifier gain, and R_{shunt} is the shunt resistor (0.007Ω).

To handle noise from switching transients and ensure a clean signal, each channel maintains a sliding window buffer of the 10 most recent samples. The average of this buffer is computed each cycle to filter out high-frequency fluctuations. The filtered phase currents are then summed in magnitude to produce an estimate of total current draw

$$I = |phaseB| + |phaseC|$$

This total current is stored in the shared variable *g_fTotalCurrent*, which is accessed by other system modules, including the graphical display and control layers. To prevent unsafe operating conditions, the system checks whether the current exceeds a defined safety threshold (e.g., 800 mA).

If so, a binary semaphore (*xEstopSem*) is used to trigger the emergency stop flag *estop_setting*. This semaphore protects concurrent access between the sensing task and other modules that respond to fault conditions, such as a motor controller. Integration was handled using the TivaWare driver library. The ADC and GPIO drivers were used to configure analog inputs.

2.2.2). Speed Sensing

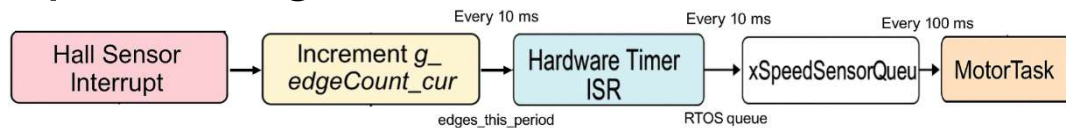


Figure 2.3: Diagram of the RTOS speed sensing, showing the sequence from Hall sensor interrupts to motor system via timer ISR and FreeRTOS queue

Speed sensing was handled and integrated with the motor system using hardware interrupts and FreeRTOS queue mechanisms for task-interrupt communication. The Hall sensor interrupt incremented a global edge count (*g_edgeCount_cur*) on each rising or falling edge, capturing every Hall state transition.

A 100 Hz hardware timer (Timer3A) triggered an ISR every 10 ms. This ISR captured the periodic hall sensor interrupt count, which was used to calculate the number of transitions in that interval (*edges_this_period*), and pushed it into the RTOS queue *xSpeedSensorQueue* using FreeRTOS function *xQueueSendFromISR()*. The queue was configured with fixed buffer size of 10 and was used to pass discrete speed samples to the main motor control task without blocking the hall sensor ISR.

The motor control task (**prvMotorTask**) receives the samples, and uses them to compute a moving average of edge counts, converted into RPM using the following calculation:

$$RPM = \frac{(\text{mean edges})}{10} \times \frac{msPerMinute}{EdgesPerREV \times timerPeriod[ms]}$$

This structure ensured smoothed, noise-handled speed measurements while retaining responsive to changes in motor behaviour. Synchronisation between the timer ISR and MotorTask was safely managed by FreeRTOS queue mechanism, ensuring that speed sensing remained decoupled from control computation, enabling robust inter-task communication within RTOS design.

2.2.3). Light Sensing

Our EV light sensor subsystem is implemented with the OPT3001 ambient light sensor. The system is implemented with an RTOS structure, using a producer-consumer pattern where a dedicated sampling task reads and processes light data, before passing it to the display task with queue-based communication. The light sensing functionality is implemented with a synchronised task structure. The OPT3001 sensor task is implemented in the 'vSensorSamplingTask' function. It runs with a higher priority (*tskIDLE_PRIORITY + 3*) and a 100ms task delay, sampling at 10Hz. To ensure measurement stability, the five-point moving average filter is applied to the raw data to reduce noise while maintaining responsiveness.

A queue (*xSensorQ*) has been implemented to handle inter-task communication, passing *SensorMessage* structures that contain raw and filtered lux values. To manage synchronisation and resource access, an I2C mutex is implemented to manage I2C access with the acceleration task.

Additionally, an event group (xEvt) manages any notifications when light levels exceed the low (50 lux) threshold.

The light sensor task uses the I2C driver for sensor communication and the OPT3001 driver for sensor configuration, data reading and conversion. The light task is integrated with the display to show a dynamic plot of light intensity on the screen, along with a day/night indicator that changes when light levels drop below 5 lux.

2.2.4). Vehicle Body Acceleration Sensing

Within our EV controller, the acceleration sensor subsystem is crucial in detecting any sudden crash events and allowing for e-stop interrupts. Within our RTOS design, the acceleration sensing task is implemented in the 'vAccelTask' function with a 10ms task delay, executing at a 100 Hz sampling rate. This sensor task is prioritised just above the idle task (tskIDLE_PRIORITY + 1) with a dedicated 256-word stack.

The Bosch BMI160 acceleration sensor is integrated with the TivaWare peripheral driver library to help manage I2C communications and hardware interactions. During initialisation, the 'bmi160Init' function performs a soft reset and configures the sensor in normal mode with a $\pm 2g$ measurement range. This configuration allows for full chip operation and accelerometer sampling, with the sensor's maximum precision. Its operation is then verified by the 'bmi160Test' routine, where it validates the chip ID of 0xD1. Once the sensor is initialised, six bytes of raw data (16 bits per axis) is read on the I2C2 bus using the 'bmi160ReadAccel' function. It then converts the raw data into physical units (m/s^2) using 'bmi160ConvertAccel' and summates the acceleration from all three axes to calculate the absolute acceleration.

The I2C2 is shared with the light sensing task on PN5 and PN4, and to avoid race conditions a mutex is initialised via 'vInitI2CMutex'. This module was designed to allow for results to be displayed on the user interface at a later stage.

2.3). User Interface

The UI developed by Rory provides a real-time interface for controlling and monitoring the EV system via the Tiva TMC1294NCPDT microcontroller, using the BOOSTXL-K350QVG-S1 LCD display and resistive touch screen.

2.3.1). Approach to Design:

The approach to designing the UI system prioritizes reactivity, usability, and real-time virtual data integration, as specified by the manufacturer. The UI was divided into two panels, the Control panel for status observation and user inputs for motors and thresholds, and the Plots panel for displaying sensor data in a visually appealing way. The Controls panel features the motor start/stop radio buttons, an RPM slider (0-5000 RPM), a real-time Clock, threshold sliders for maximum current and acceleration, a motor status (on-screen and LED), an E-Stop acknowledgment button, and a Day/Night Indicator on screen. The Plots panel presents the line graphs for the filtered sensor data for the motor power, motor speed, light levels, and absolute acceleration, allowing up to 10 seconds of previous data to be displayed and certain graphs to be turned off via a checkbox button.

The design uses the ‘grlib’ graphics library to render widgets, such as buttons, sliders, and canvases, allowing a clear and consistent layout. FreeRTOS tasks and queues were used to help integrate the UI subsystem with the sensor and motor control subsystems, ensuring updates without blocking or stopping critical operations. A visual representation of these panels can be seen in Figure 2.1 below, showing the control and plot panels in certain scenarios. The first panel is the Control panel when Estop is activated. The second panel is the Control panel when the system is running, and the Third panel is the Control panel when the system is stopped. The fourth panel is the Plots panel with all plots, whilst the fifth panel is the Plots panel with a deselected plot.

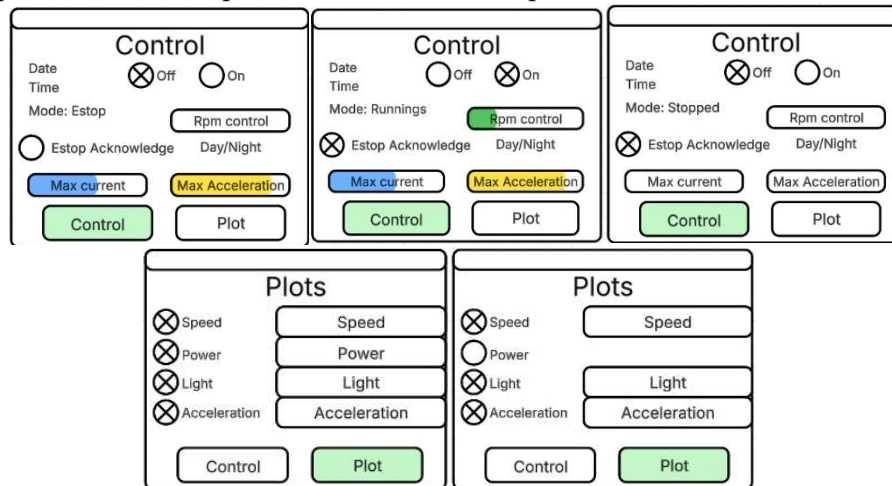


Figure 2.4: Panel Layout

2.3.2). RTOS Task Structure, Task Communication & Synchronization:

The UI subsystem works within the FreeRTOS task ‘prvDisplayDemoTask’, which interacts with the other tasks allowing for real-time updates. Figure 2.2 displays the RTOS task structure and how each task communicates with the UI subsystem. The ‘main.c’ file is used to initialise hardware and create tasks for the multiple subsystems. The ‘prvDisplayDemoTask’ receives three types of sensor data; the light sensor data via a queue and the motor and speed sensor data via protected semaphores. The UI subsystem also sends to the ‘prvMotor’ task, such as the new desired RPM.

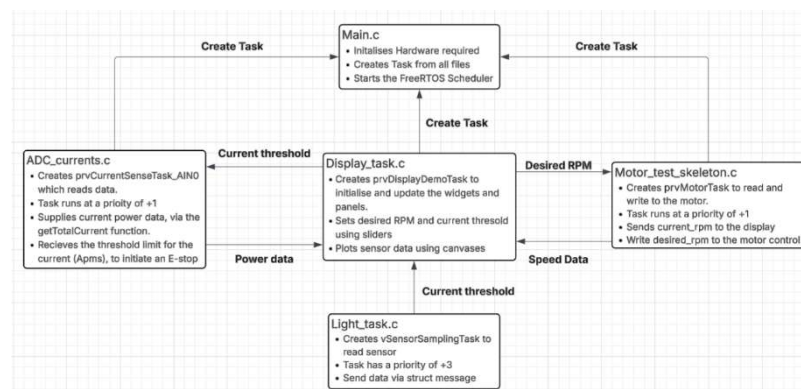


Figure 2.5: RTOS Task Structure

2.3.3). Important Issues and Choices:

A significant challenge was sorting out potential conflicts between certain parts of the overall system, in particular the touchscreen driver and the motor current sensing. In the provided examples, both sections used ADC0, which could lead to a system failure. This problem was fixed by changing the current motor sensing to ADC1 and ADC0 stays the same, being used for touch input. One design choice that was made was to have the plots display the last 10 seconds of the sensor data to allow for sufficient time to visually analyse the sensor data to be able to identify errors in systems or monitor certain parameters. Another design choice that was made was to have a different way of navigating panels. In the example given, the two navigation buttons were either previous or next, which switched panels in a particular order. This navigation system was updated to allow navigation between two panels using a new function, 'SwitchToPanel', which would change the panel for the current panel to the requested panel. This function would allow for navigation between more than two panels and would not navigate between panels in a particular order. This function would be useful if more panels, such as a setting or camera panel, were required.

2.3.4). Drivers, Modules, and External Libraries:

The two drivers that interact with the microcontroller hardware are the 'touch.h' and the 'Kentec320x240x16_ssd2119_spi.h' files, which manage the touchscreen input and output. The 'touch.h' file controls the touchscreen input with the ISR routine event to the widget tree, whilst the 'Kentec320x240x16_ssd2119_spi.h' file is used to initialise the interface, rendering a 320x240 pixel display. The only external library used was the 'glib' graphics library, which allowed for easier construction of the UI subsystem including its widgets and panels. Within the 'glib' graphic library, there are many modules that are used, which include 'glib.h', 'widget.h', 'canvas.h', 'slider.h', 'checkbox.h', 'container.h', 'pushbutton.h' and 'radiobutton.h', which are used for widgets sliders for control and thresholds and buttons for control and navigation. The 'ustdlib.h' is used for string formatting, whilst the 'xSensorQ' is a queue that interacts with the sensor subsystem for the light sensor data.

2.3.5). Relationship to Hardware and Software Platforms:

The UI utilises the Tiva microcontroller interface for display communication and GPIO interrupts for touchscreen events. To meet the RTOS requirements, FreeRTOS is used to ensure real-time scheduling, with the UI task running parallel to other subsystem tasks. The touchscreen needed to be carefully integrated, which was assisted by using provided drivers, such as 'touch.h' and 'Kentec320x240x16_ssd2119_spi.h'. While the 'glib' graphics library increases memory usage, it is essential for usability as it supplies a strong framework for UI development.

3). Results

3.1). Motor Control

The motor system met most functionality requirements, including safely starting the motor, and updating the speed via input rpm setting in accordance with safe acceleration and deceleration limits. The e-stop system is implemented successfully within the motor system, however integration of this function failed. The following section details the validation procedure and discussion of results. The serial plotting utilised VS Code extension serial-plotter (which I would recommend for simple, arm64-friendly, vs-code integrated serial plotting).

The GUI was used to set a desired speed of 2000 RPM, while a Serial Plotter logged the actual and desired RPM at 10 Hz (100ms period). After 15 seconds, a counter triggered a hard-coded E-stop.

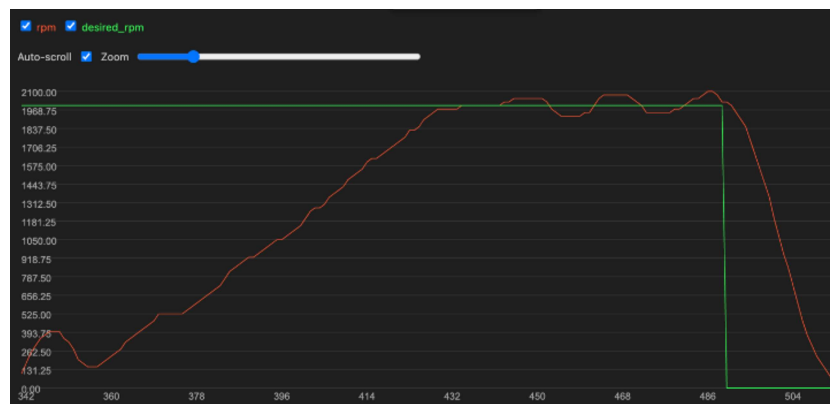


Figure 3.1: Serial Plot showing acceleration to 2000 RPM and error before E-stop activation.
Note: x-axis is in 100 ms scale (100 = 10s).

During the normal motor control response, the system steadily increased from 0 to 2000 RPM in approximately 8 seconds, validating the desired acceleration rate of ~ 250 RPM/s — within the 500 RPM/s specification. Once the desired RPM was reached, the actual RPM demonstrated a control error of $< 6\%$, well within the $\pm 20\%$ requirement.

Although this approach does not implement a PI or closed-loop error feedback control, it meets system requirements by staying well within the required $\pm 20\%$ error. However, future development may benefit from switching to a PI controller for more precise speed control performance.



Figure 3.2: Serial plot showing deceleration after E-stop trigger.

Emergency stop performance was validated via an independent hard-coded trigger. As seen in figure 3.2, the desired RPM signal dropped instantaneously to 0. The actual RPM followed a linear deceleration and reached zero in just over 2 seconds. The exact time taken was approximately 2.3s, however, the additional response time is likely due to external factors relating to use of the motorlib **motorStop()** API function and momentum of the motor. The white dotted lines indicate the point at which the **motorStop()** function was called ($RPM \leq 250$). This consieration demonstrated a deceleration from 2000 RPM to 250 RPM in $\approx 1.8s$. This corresponds to a deceleration rate of approximately 975 RPM/s, validating that the E-stop function very closely obeyed the 1000 RPM/s deceleration target.

Though it was not event-driven, the timer-based simulation validated that the motor could be stopped within the E-stop requirements. Future work would complete the intended integration with shared semaphores with the power and acceleration sensing tasks to ensure the system can respond to genuine emergency inputs.

3.2). Sensing

3.2.1) Power sensor Filtering

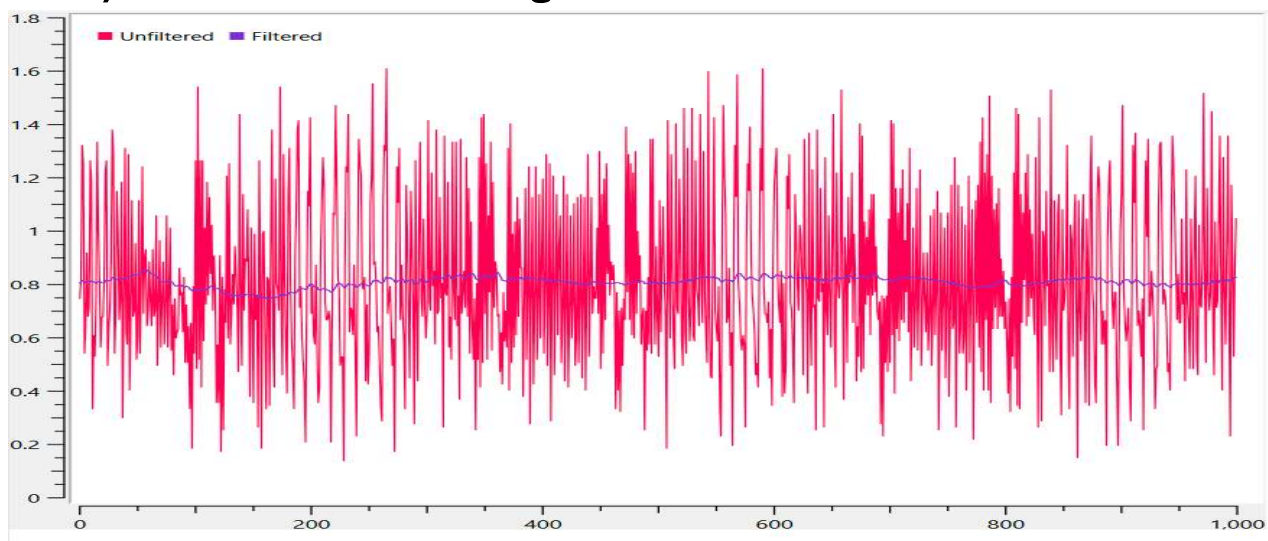


Figure 3.3: Comparison of unfiltered (red) and filtered (blue) power readings.

The effectiveness of the implemented digital filtering strategy was evaluated by comparing the unfiltered and filtered power signals over time, as illustrated in Figure 3.3. The unfiltered signal (shown in red) exhibits pronounced fluctuations and high-frequency noise, resulting from instantaneous disturbances and switching transients inherent in the motor's PWM operation.

To mitigate these effects, a sliding window filter of ten samples was applied to the phase current signals prior to computing the total power. The resulting filtered power signal (shown in blue) demonstrates significantly improved stability. While it retains the underlying trend and variation of the true power consumption, it suppresses short-duration spikes and erratic noise present in the raw measurements.

This filtering process enables more accurate and reliable real-time power monitoring. It enhances the interpretability of power data for diagnostic display purposes and ensures consistent behaviour in safety-critical functions such as overcurrent protection, where excessive noise could otherwise cause false positive fault triggers.

3.2.2) Speed Sensor Filtering

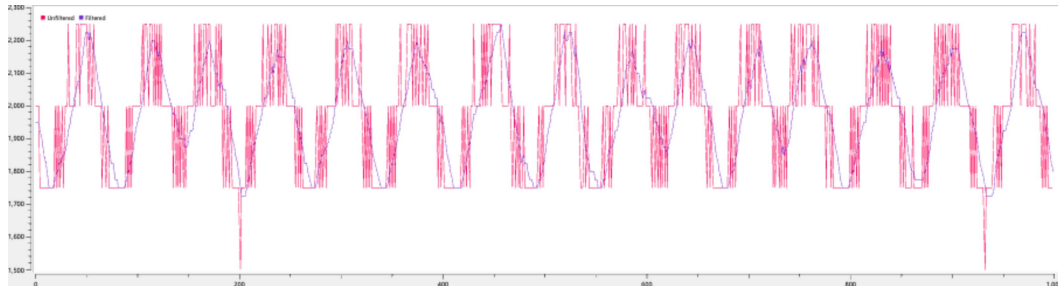


Figure 3.4: Comparison of unfiltered (red) and moving-average filtered (blue) RPM estimates

The accuracy and stability of the speed sensing implementation was validated by comparing the unfiltered and filtered edge count signals over time, as seen in Figure X. The unfiltered data (red) showed frequent spikes and instability, resulting in high-noise output data.

The application of a 10-sample moving average effectively suppressed these fluctuations. The filtered signal (blue) retained the underlying frequency trend while discarding transient noise, resulting in smoothed and stable RPM estimates.

3.2.3) Light Level Sensor

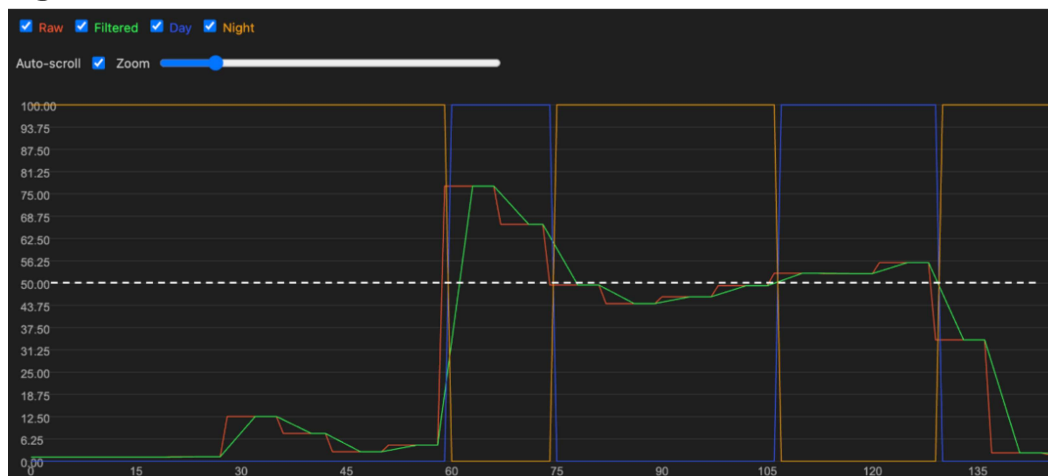


Figure 3.5: Real-time plot of raw and filtered OPT3001 light measurements with visualised day/night transitions at threshold=50 lux.

The performance of the light sensor subsystem under controlled lighting changes is visualised using a serial plot in Figure 3.5. The system successfully captured variations in light, with clear improvements of the raw (red) readings using a five-point moving average filter, producing the filtered (green) readings. This significantly smoothed noise present in the raw data, while retaining responsive during sudden lighting changes.

A day/night indicators functionality was validated using a visual 50-lux threshold (white dotted line) for improved visualisation. While filtering of Lux readings was important for smooth visualisation of ambient lighting, raw readings were using for the flagging of these day/night conditions. The system reliably asserted the day event flag (orange) when raw lux rose above this threshold and asserted the night event flag (blue) upon return to dimmer lighting. This behaviour was confirmed in multiple cycles, accurately reflecting the systems handling of lighting transitions.

3.2.4) Vehicle Acceleration Sensor

The acceleration sensor was tested via UART. During initialization, the sensor was set to normal mode with a $\pm 2g$ range and its chip ID verified. However, it unexpectedly entered suspend mode, reporting a constant acceleration across all axes. Although raw data conversion produced a steady 9.21 m/s^2 —close to gravity—the sensor failed to update, leading to its exclusion from the final design. A deeper review of the BMI160 datasheet and its initialization procedures is needed to prevent this issue in future implementations.

3.3). User Interface

The UI subsystem successfully demonstrated many functional requirements on the Tiva TMC1294NCPDT microcontroller. On the Control panel, the RPM slider adjusted the desired motor speed with the correct values being sent to the Motor Task. The real-time clock showed the correct date and time, even updating if the user was on the Control or Plot panel. On the Plot panel, the sensor data successfully visualised the sensor data for motor power, motor speed, and light levels, which display data from the last 10 seconds' worth of data. Figure 3.6 shows a photo displaying that the plots work and displays their respective sensor data.

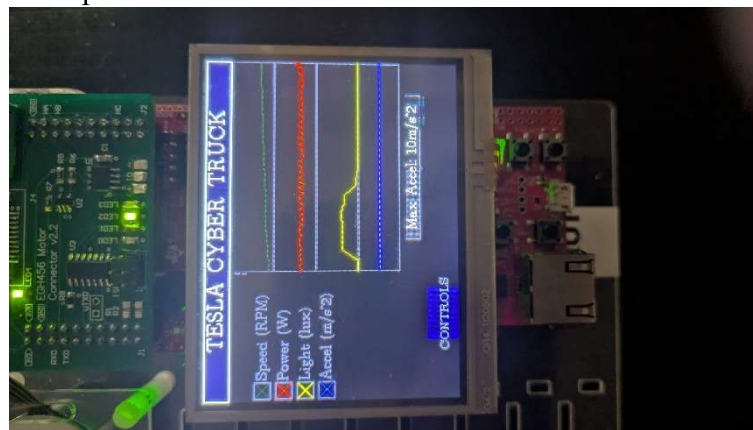


Figure 3.6: Working Plots Panel

However, some requirements were either not able to be implemented visually or were implemented visually but were unable to integrate with other subsystems. The start/stop button was visually implemented but was unable to start nor stop the motor due to subsystem integration. The sliders for current and acceleration thresholds were also visually implemented but unable to communicate with other subsystems. The system Status LED configurations were not implemented due to time and priority constraints, as the LED would be easy to implement once all three types of system statuses were implemented and shown to be working via the status text on the screen. The system status text overwrote previous messages, needing a proper updating method like the time and date.

The E-Stop acknowledgment button was implemented but drawn over by another widget, particularly when the time updates. The absolute acceleration plot was visually implemented but no sensor data was able to be plotted. Although the acceleration plot did not show sensor data, it did work based on the max acceleration slider in the control panel. Fixing this issue would be very easy as it would only be changing the input for the plot from the slider input to the sensor data. The Day/Night indicator did work in real-time but did not change the indicator on the control panel unless the panels switched from control to plot and then back to control. These problems highlight the requirement for better communication between tasks, such as start/stop buttons not being able to communicate to the motor task to either run or stop the motor. Another problem that was highlighted was the requirement for more testing and evaluation of the widget and panel rendering order, as some widgets and texts were unable to be seen due to being drawn over. Of course, getting the acceleration, E-stop, and LED integration would have improved overall performance.

Real hardware testing included using the software on the Tiva microcontroller, having physical interaction with the touchscreen, and using UART. These methods for testing were used as touchscreen interactions would confirm system responsiveness and navigation whilst the UART would verify touchscreen interactions and print messages, such as 'Switch to panel #' or 'setting RPM slider value to ##'. UART was also used to confirm that sensors were initialised and reading data.

4). References

Ref	Section	Topics	Pages	Document Title	URL
1	2.2.3 3.2	Device Functional Modes, Programming & Register Maps	11-21	OPT3001 Ambient Light Sensor (ALS)	https://www.ti.com/lit/ds/symlink/opt3001.pdf
2	2.2.4	Sensor mode, range initialisation & functionality	13-15 58-59	BMI160 Small, low power inertial measurement unit Data sheet	https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmi160-ds000.pdf
3	2.3.2 2.3.3 2.3.4 2.3.5	ADC, UART & Hardware Interface	742-814 1053-1225 1275-1355	Tiva TM4C1294NCPDT Microcontroller Data Sheet	https://www.ti.com/lit/ds/symlink/tm4c1294ncpdt.pdf
4	2.3.3 2.3.4 2.3.5	GPIO, sensors & UART	9-10 20-23	Tiva C Series TM4C1294 Connected Launchpad Evaluation Kit User's Guide (EK-TM4C1294XL)	https://www.ti.com/lit/ug/spmu365c/spmu365c.pdf
5	2.3.1 2.3.2 2.3.4	Display and Motor Control Integration	4-6	BOOSTXL-K350QVG-S1 QVGA Display BoosterPack Plug-In Module	https://www.ti.com/lit/ug/slau601b/slau601b.pdf
6	2.3.2 2.3.4 2.3.5 2.3.6	Canvas Widget, Checkbox Widget, Push Button Widget, Radio Button Widget, Slider Widget, Colour Reference	6-7 85-96 97-114 115-130 199-260 269-274	TivaWare Graphics Library User's Guide	https://www.ti.com/lit/ug/spmu300e/spmu300e.pdf
7	2.1, 2.2.3	FreeRTOS Queues, FreeRTOS ISR-Queue, FreeRTOS Semaphores, FreeRTOS Events	97-102, 187-190, 163-168, 225-228	Mastering the FreeRTOS real-time kernel: A hands-on tutorial guide (Version 1.1.0)	https://www.freertos.org/Documentation/RTOS_book.html